



UNIVERSIDADE DE COIMBRA
Faculdade de Ciências e Tecnologia
Departamento de Engenharia Informática

Estratégias Algorítmicas
Exame Normal – 15 de junho de 2026

Nome: _____ N° de estudante: _____

16 pontos no total, 2 horas, sem consulta. Formulário disponível na última página.

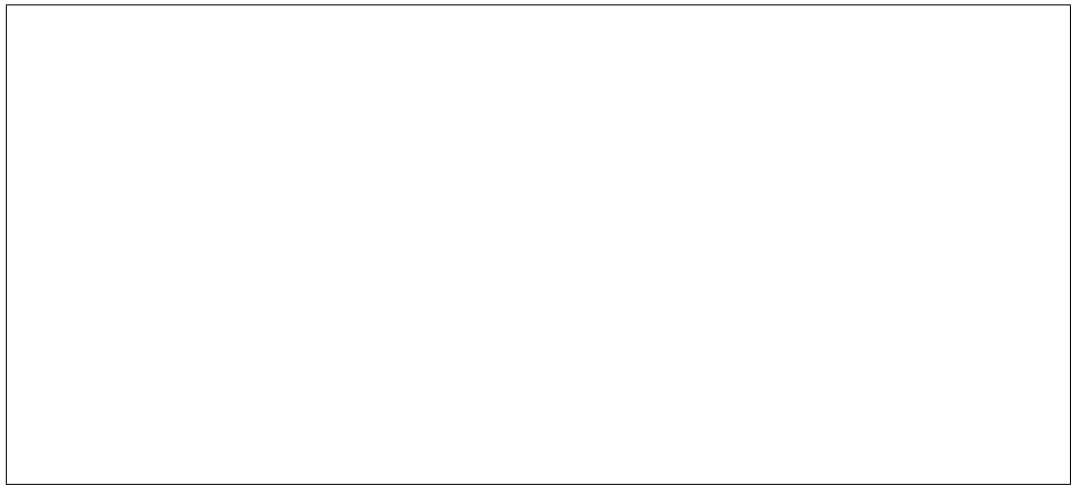
1. Considere a seguinte recorrência para $i = 0, \dots, n$, $j = 0, \dots, n$ e $c_i \geq 0$:

$$C(i, j) = \begin{cases} 0 & \text{se } i < 1 \text{ or } j < 1 \\ c_i + \min\{C(i-1, j), C(i, j-1), C(i-1, j-1)\} & \text{caso contrário} \end{cases}$$

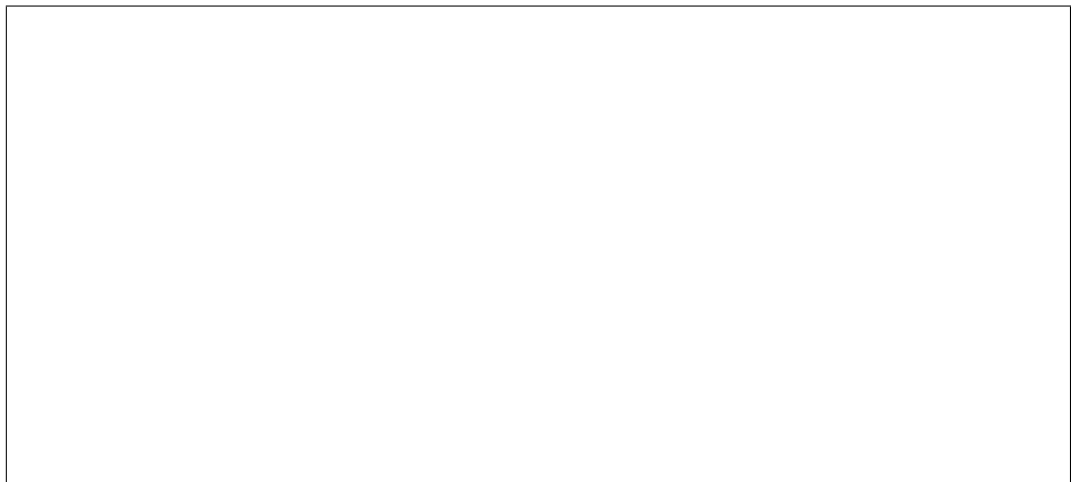
Tendo em conta que o valor que pretende obter é retornado por $C(n, n)$, apresente o pseudo-código de um algoritmo de programação dinâmica ascendente (*bottom-up*) que explore a recorrência acima para obter esse valor e discuta a sua complexidade computacional. (3 pontos)

2. O algoritmo de Bellman-Ford (ver $BellmanFord(G, s, t)$ no formulário) recebe como entrada um grafo ponderado $G = (V, A, w)$, onde V representa o conjunto de vértices, A o conjunto de arco e w uma função de peso que associa a cada arco $(u, v) \in A$ um valor $w(u, v)$ correspondente ao custo de percorrer o arco do vértice u para o vértice v . Assumindo a inexistência de ciclos negativos alcançáveis a partir de s , o algoritmo calcula e devolve o custo mínimo de um caminho entre o vértice de origem s e o vértice de destino t .

- a) Assuma que todas os arcos do grafo G têm custo não negativo. Nestas condições, é possível reduzir o número de iterações efetivamente executadas pelo ciclo exterior do algoritmo de Bellman-Ford, embora a sua complexidade assintótica no pior caso permaneça inalterada. Indique as alterações (mínimas) a efetuar ao pseudocódigo apresentado no formulário e justifique por que motivo essas alterações permitem terminar o algoritmo mais cedo em alguns casos, sem alterar o pior caso. (1.5 pontos)



- b) No pseudocódigo apresentado na última página, assume-se que $d(v) = \infty$ para os vértices ainda não alcançados. Numa linguagem de programação como C++ ou Java, não é possível representar diretamente o valor ∞ , sendo necessário utilizar uma constante suficientemente grande para esse efeito. Contudo, a utilização desse valor pode originar situações de *overflow* nas operações efetuadas nas linhas 6, 7, e 9 do pseudocódigo. Indique e justifique as alterações necessárias ao pseudocódigo da função $BellmanFord$ para garantir que não ocorrem situações de *overflow* durante a sua execução. (1.5 pontos)



3. Considere o seguinte pseudocódigo de um algoritmo que permite calcular o valor de x^n para dois inteiros não-negativos n e x .

Function $pot(x, n)$

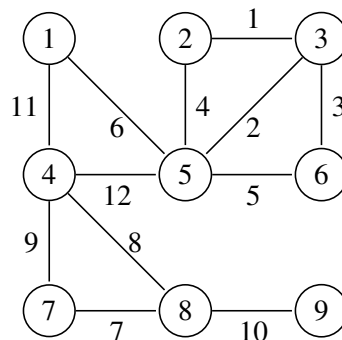
```
1: if  $n = 0$  then  
2:   return 1  
3: if  $n$  é ímpar then  
4:   return  $x \times pot(x, n - 1)$   
5: else  
6:   return  $pot(x, n/2) \times pot(x, n/2)$ 
```

- a) Desenhe a árvore de recursão para $pot(2, 6)$. (1 ponto)

- b) Prove por indução matemática que $pot(x, n) = x^n$ para todo $n \geq 0$. Indique explicitamente o caso base, a hipótese de indução, e os passo(s) indutivo(s). (1.5 pontos)

- c) É possível aumentar a eficiência deste algoritmo evitando a duplicação das duas chamadas recursivas na linha 6. Modifique o pseudocódigo de forma a eliminar essa redundância e tornar o algoritmo mais eficiente. (1.5 pontos)

4. Considere o seguinte grafo.



- a) Desenhe a árvore geradora mínima (à esquerda) e o grafo da estrutura de dados *union-find*, sem o passo de compressão de caminho (à direita), recorrendo ao algoritmo de Kruskal. Quando necessário, ligue a raiz da árvore com menor altura à raiz da árvore com maior altura e, em caso de empate, escolha, como raiz, o vértice que apresentar a etiqueta com o menor valor. (1.5 pontos)

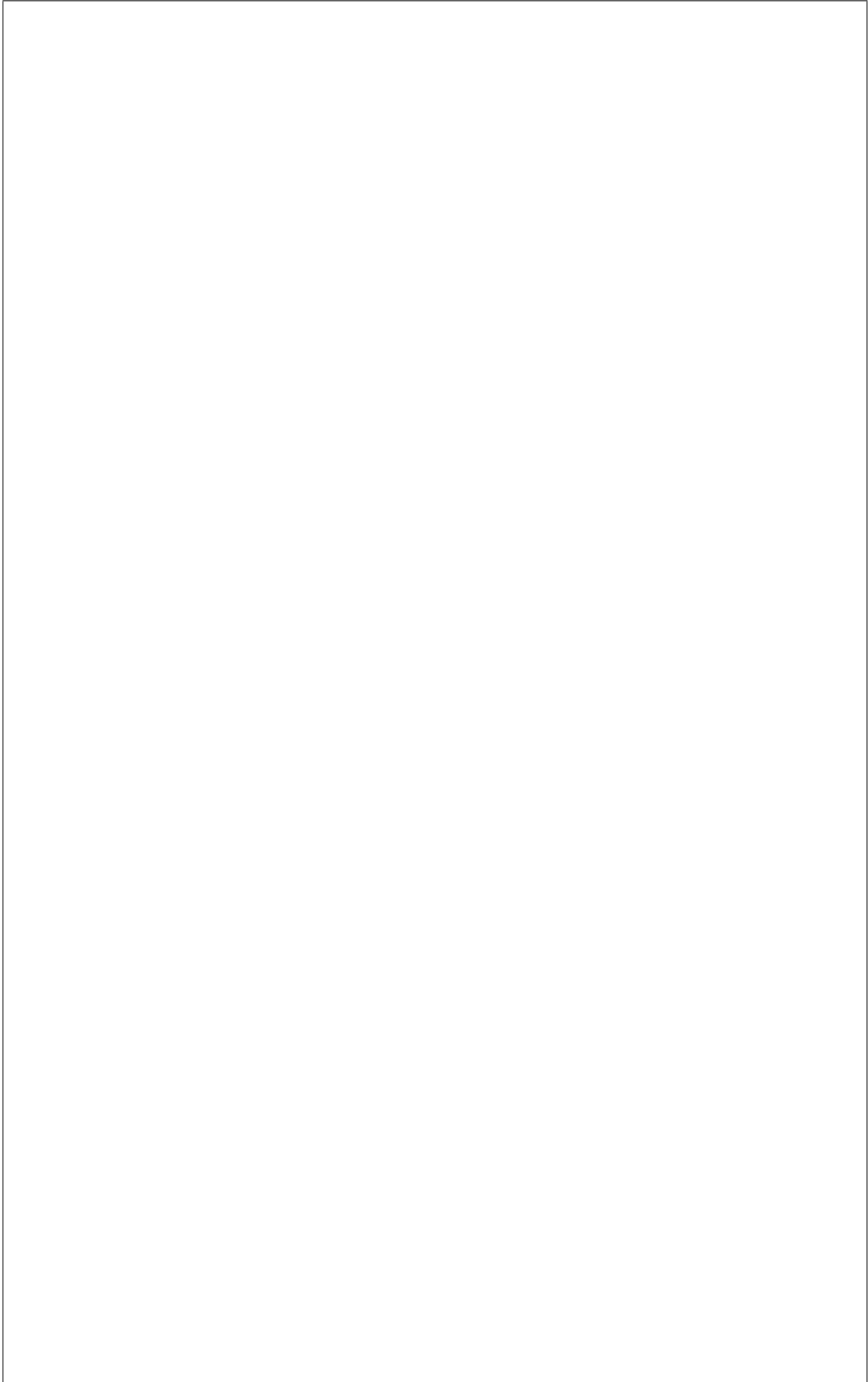


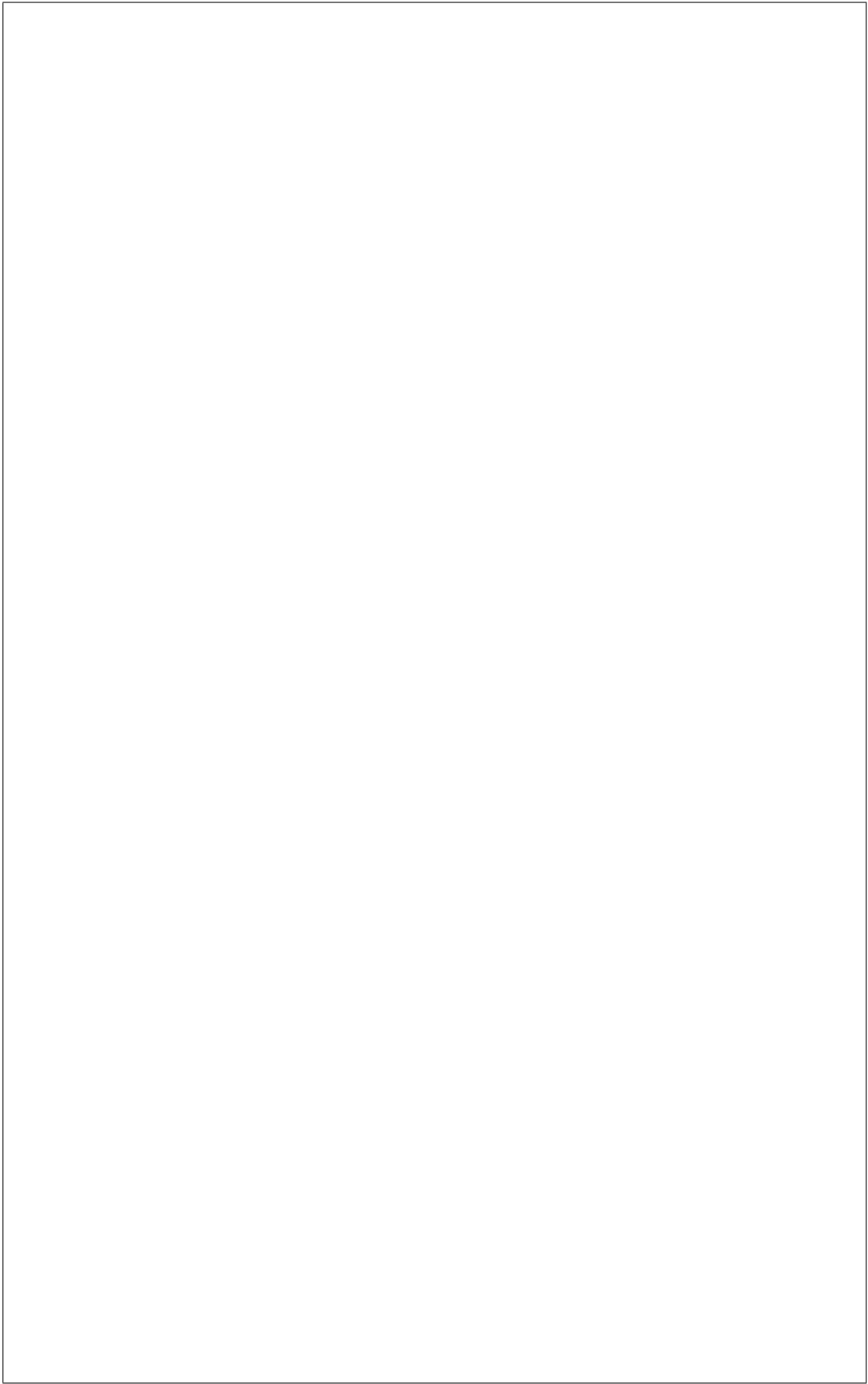
- b) O algoritmo da função $find(a)$, apresentado na última página, implementa a técnica de compressão de caminho. Escreva uma versão da função que preserve a sua funcionalidade, mas sem efetuar compressão de caminho. (1.5 pontos)

5. Considere o seguinte problema: distribuir n pessoas por n lugares numa mesa redonda, respeitando as seguintes restrições:

- As pessoas A e B não podem ocupar lugares adjacentes;
- As pessoas C e D devem ocupar lugares adjacentes.

Note que a pessoa que ocupa o n -ésimo lugar está adjacente à pessoa que ocupa o primeiro lugar. Descreva, em pseudocódigo, um algoritmo de *backtracking* que construa a distribuição lugar a lugar, eliminando soluções parciais que não possam conduzir a uma solução válida. O algoritmo deve reportar SIM se existir uma distribuição válida e NÃO caso contrário. Comente o pseudocódigo apresentado, identificando os casos base e os passos recursivos. Indique também as estruturas de dados necessárias para armazenar os dados de entrada e a solução (parcial). (3 pontos)





Graph Algorithms

Function *Dijkstra*(G, s, t)

```
1:  $Q$  contains all vertices in  $G$ 
2: for each vertex  $v \in V \setminus \{s\}$  do
3:    $d(v) = \infty$ 
4:  $d(s) = 0$ 
5: while  $Q \neq \emptyset$  do
6:    $u$  is the closest vertex to  $s$  in  $Q$ 
7:   vertex  $u$  is removed from  $Q$ 
8:   if  $u = t$  then
9:     break
10:  for each arc  $(u, v) \in A$  do
11:    if  $d(v) > d(u) + w(u, v)$  then
12:       $d(v) = d(u) + w(u, v)$ 
13: return  $d(t)$ 
```

Function *BellmanFord*(G, s, t)

```
1: for each vertex  $v \in V \setminus \{s\}$  do
2:    $d(v) = \infty$ 
3:  $d(s) = 0$ 
4: for  $i = 1$  to  $|V| - 1$  do
5:   for each arc  $(u, v) \in A$  do
6:     if  $d(v) > d(u) + w(u, v)$  then
7:        $d(v) = d(u) + w(u, v)$ 
8: for each arc  $(u, v) \in A$  do
9:   if  $d(v) > d(u) + w(u, v)$  then
10:    return negative cycle found
11: return  $d(t)$ 
```

Function *FloydWarshall*(G, D)

```
1: for  $i = 1$  to  $|V|$  do
2:    $D[i, i] = 0$ 
3: for each arc  $(i, j) \in A$  do
4:    $D[i, j] = w(i, j)$ 
5: for  $k = 1$  to  $|V|$  do
6:   for  $i = 1$  to  $|V|$  do
7:     for  $j = 1$  to  $|V|$  do
8:        $D[i, j] = \min(D[i, j], D[i, k] + D[k, j])$ 
9: return  $D$ 
```

Function *AP*(v)

```
1:  $low[v] = dfs[v] = t$ 
2:  $t = t + 1$ 
3: for each edge  $\{v, w\} \in E$  do
4:   if  $dfs[w]$  has no value then
5:      $parent[w] = v$ 
6:      $AP(w)$ 
7:      $low[v] = \min(low[v], low[w])$ 
8:   if  $dfs[v] = 1$  and  $dfs[w] \neq 2$  then
9:      $v$  is a (root) AP
10:  if  $dfs[v] \neq 1$  and  $low[w] \geq dfs[v]$  then
11:     $v$  is a (non-root) AP
12:  else if  $parent[v] \neq w$  then
13:     $low[v] = \min(low[v], dfs[w])$ 
```

Function *SCC*(v)

```
1:  $low[v] = dfs[v] = t$ 
2:  $t = t + 1$ 
3:  $push(S, v)$ 
4: for each arc  $(v, w) \in A$  do
5:   if  $dfs[w]$  has no value then
6:      $SCC(w)$ 
7:      $low[v] = \min(low[v], low[w])$ 
8:   else if  $w \in S$  then
9:      $low[v] = \min(low[v], dfs[w])$ 
10: if  $low[v] = dfs[v]$  then
11:    $C = \emptyset$ 
12:   repeat
13:      $w = pop(S)$ 
14:      $push(C, w)$ 
15:   until  $w = v$ 
16:    $push(SCC, C)$ 
```

Function *makeSet*(V)

```
1: for each vertex  $i \in V$  do
2:    $set[i] = i$ 
3:    $rank[i] = 0$ 
```

Function *union*(a, b)

```
1:  $link(find(a), find(b))$ 
```

Function *find*(a)

```
1: if  $set[a] \neq a$  then
2:    $set[a] = find(set[a])$ 
3: return  $set[a]$ 
```

Function *link*(a, b)

```
1: if  $rank[a] > rank[b]$  then
2:    $set[b] = a$ 
3: else
4:    $set[a] = b$ 
5: if  $rank[a] = rank[b]$  then
6:    $rank[b]++$ 
```